

Organ-on-Chip Image Evaluation Audit

A reproducible case study of acquisition-context overlap and split sensitivity

Yan Su

26 September 2026

Submission category: Tool & Platform

Team: Yan Su (individual entrant)

Contents

1	Abstract	3
2	Research problem and intended use	4
3	Data, provenance, and permission	5
4	Audit design	6
5	Image features and frozen model protocol	7
6	Measured results	8
7	Software artifact and real workflow	10
8	Reliability, limitations, and application value	11
9	Reproducibility and dependency disclosure	12
10	Prior work, originality, and responsible disclosure	15
11	References	15
12	Appendix A: denominators and decision gates	16
13	Appendix B: numeric audit checks	17

1 Abstract

An image-quality model can score well on a held-out folder while failing to answer the question an organ-on-chip researcher actually asks: will it work on a new acquisition context? We built an offline audit that joins image identifiers, expert quality labels, acquisition groups, train/validation/test assignments, and test predictions. It reports group overlap, class denominators, confusion matrices, false alarms on good images, and a same-image comparison between two evaluations. The report is generated from records and includes the records and calculations in JSON.

We demonstrate the tool on 3,072 real organ-on-chip brightfield images and expert good/bad labels from the public Zenodo dataset by Movčana and colleagues [1, 2]. The original ZIP’s image-level test contains 57 date-like filename prefixes; all 57 also occur in its training set. In the clean Linux CPU run, a fixed 29-feature random forest scores 0.799925 balanced accuracy on that test, while the corresponding model evaluated on a prefix-group holdout scores 0.657920. These test sets share only 151 images, so their 0.142005 difference is descriptive. On the shared images, the source-split and grouped-split models score 0.756467 and 0.690379, a difference of 0.066088. A paired percentile bootstrap over 14 prefixes gives an approximate 95% interval from -0.042091 to 0.122483. The interval crosses zero, and the two models were trained and calibrated on different rows. These observations do not establish a general optimistic bias, a causal effect of overlap, or leakage between physical chips.

The dataset does not provide independently verified physical chip IDs. Filename prefixes are therefore treated only as acquisition-context proxies. The Linux CPU result is the frozen reference for the public command; the original macOS run is retained as a historical measurement. Both runs used a byte-identical feature file but built different forest trees, while the prespecified research decisions were unchanged. The tool checks Linux outputs against the Linux reference and marks a macOS difference. The contribution is an inspectable evaluation workflow, a fully worked real-data case, and explicit limits on what each comparison can establish. It requires no paid API, GPU, proprietary model, or redistribution of the source images.

Keywords: organ-on-chip; brightfield imaging; evaluation audit; grouped holdout; reproducibility; image quality.

2 Research problem and intended use

2.1 A concrete decision at the bench

An organ-on-chip imaging team may train a classifier to send low-quality brightfield images for manual review. Before using that model, the team needs to know whether held-out images truly come from acquisition contexts absent from training. It also needs the number of good cultures that would be incorrectly flagged. A single balanced-accuracy value hides both questions.

This project provides a small audit for those decisions. Its inputs are one record per image from each evaluation: a stable image ID, a group identifier, the reference good/bad label, the split assignment, and a test prediction. The group might be a physical chip, an experiment, an imaging session, or a weaker proxy. The scientist must identify which it is from the source data. The tool computes what the records support and labels the comparison’s evidence level. It does not infer a physical chip ID from a filename.

Our worked example uses the public image-quality dataset. In it, the first six digits of each image ID resemble a date. We use that prefix to ask whether the source folder split reuses acquisition context across training and test. The prefix is not a verified chip identifier. Accordingly, the user-facing report says “acquisition-context overlap,” not “chip leakage.”

2.2 Why this is an AI-for-life-science problem

The useful unit of generalization depends on how the life-science data were collected. Many images from the same chip or imaging session are not equivalent to independent future experiments. The dataset at hand has expert quality labels and actual brightfield images, so it supports a real, reproducible image-evaluation case. It does not contain toxicity outcomes, neural connectivity labels, or drug-response ground truth; none is claimed here. The practical value of the audit is that a researcher can examine evaluation units and false alarms before interpreting a model score as a deployment result.

The prior classification study using this same source data reports a single image-level split and identifies stability across splits as future work [3]. Our contribution is a different question: expose grouping assumptions, test-set composition, and same-image prediction changes using a reusable record contract. We neither reproduce its InceptionV3 model nor claim superiority over it.

3 Data, provenance, and permission

3.1 Source material

The [Organ-on-a-Chip Image Dataset](#) [1] contains 3,072 PNG images, a spreadsheet of experimental metadata and expert image-quality labels, and original train/validation/test folders. The accompanying data descriptor explains image capture, cultures, and expert labeling [2]. The original table’s quality code is 1 for good and 2 for bad, as checked against the source ZIP folders. We map good to 0 and bad to 1 in the audit records. Six cell types occur in the table. No patient-level or private clinical data are used.

The spreadsheet is 119,712 bytes and is checked by SHA-256 before parsing. The image ZIP is 6,710,767,405 bytes; its downloaded content matched the Zenodo MD5 8f7e058996203d48eb03b2d86c0a2e4d. Extracted image IDs are checked one-to-one against the 3,072 table rows. The source split is read from the ZIP64 central directory via range requests, then every mapped ID, folder label, and cell type is reconciled to the spreadsheet. The source directory mapping has 2,130 training, 286 validation, and 656 test images. The code stores raw images and derived feature records locally; the public code repository does not redistribute them.

3.2 License boundary

The Zenodo record metadata lists CC BY 4.0 for its files [1]. The associated descriptor refers to the dataset as CC-BY-SA without a version [2]. We report both statements. The downloadable images and their adapted forms are not included in our code repository, so the discrepancy is not resolved by silently republishing under either license. Users follow the source record and its attribution terms when obtaining the data. Our own code is MIT licensed. NumPy uses BSD-3-Clause; Pillow describes its license as MIT-CMU; SciPy and scikit-learn use BSD-3-Clause [4-6, 8]. The source of each external component is listed in the references.

3.3 Evaluation groups

For the alternative holdout, define `prefix = image_id[:6]`. To make the split reproducible without choosing groups based on their labels, compute `SHA256("26" + prefix)` as an integer, then take the remainder modulo 100. Buckets 0-19 are test, 20-29 validation, and 30-99 training. This yields 38/6/15 prefixes and 2,084/252/736 images respectively. The grouped test has 376 good and 360 bad images. The original source test has 365 good and 291 bad. All 57 date-like prefixes in the source test also occur in its training data; none of the 15 grouped-test prefixes

occurs in the grouped training data.

These group counts refer to filename prefixes. They are evidence of overlap in that observed identifier. They do not count independently known physical chips, and they cannot establish whether different prefixes belong to the same chip.

4 Audit design

4.1 Record contract and validation

The reusable audit takes two JSONL files. Each line has `id`, `group`, `label`, `split`, and `prediction`; an optional `subgroup` can carry a cell type. Labels are 0 for good and 1 for bad. Predictions are 0 or 1 on test images and null on training and validation images. Each evaluation must contain one record per ID. The program rejects duplicate IDs, missing groups, invalid labels or splits, predictions on non-test rows, and conflicting labels or groups for a shared ID. It does not invent a class score when only one class is present.

For each evaluation, it counts images by split and label, computes the test confusion matrix, balanced accuracy, recalls, and good-image false-positive rate, then counts test groups also seen in training. It intersects the two test sets by exact image ID and recomputes the same measures on the shared images. A four-cell transition table records whether both models were right, only the first was right, only the second was right, or both were wrong.

Balanced accuracy is the arithmetic mean of good and bad recall. With confusion matrix rows as true good/bad and columns as predicted good/bad, write the cells as $[[GG, GB], [BG, BB]]$. Then good recall is $GG/(GG+GB)$, bad recall is $BB/(BG+BB)$, and the good-image false-positive rate is $GB/(GG+GB)$. A missing class makes balanced accuracy undefined rather than zero.

4.2 Paired uncertainty calculation

On the shared test images, we resample observed prefix groups with replacement, using the **same chosen groups** for both models. Each replicate recomputes balanced accuracy on each model’s predictions and stores the source-minus-grouped difference. We take the 2.5th and 97.5th percentiles of 2,000 valid replicates with seed 26. There are 14 prefixes in the shared set. This is an approximate descriptive interval for this dataset and this observed grouping, not a confidence guarantee across physical chips or new laboratories.

4.3 Evidence categories

The HTML and JSON distinguish three claims. Group overlap is directly observable for the supplied group IDs. The full-test score difference compares different images and is descriptive. The shared-test comparison holds test image IDs fixed, but the two fitted models still use different training rows and different validation rows and thresholds. It therefore narrows one source of confounding without isolating the cause of the difference. All conclusions remain specific to one dataset.

5 Image features and frozen model protocol

5.1 Low-cost image representation

Each source PNG is converted to grayscale and bilinearly resized to 128 by 96 pixels. The 29-dimensional feature vector contains five intensity percentiles, mean, standard deviation, Laplacian variance, mean horizontal and vertical absolute differences, a thresholded edge fraction, eight normalized histogram bins, four quadrant means, four quadrant standard deviations, and variation of row and column means. Every feature is finite-checked. This representation was selected as a low-cost baseline, not as a claim of state-of-the-art morphology recognition. Source images are only read to compute these local features; the report itself audits evaluation records.

5.2 F1: initial baselines

F1 fits a deterministic L2-regularized logistic classifier on the grouped split. Metadata-only features are cell type, seeding density, time after seeding, day, and flow rate. Missing-value medians and scaling are estimated on training rows only. The image-only model uses the 29 grayscale features; training-only scaling and fitting are followed by a validation-only threshold chosen to maximize balanced accuracy, with fixed tie rules. The constant-good baseline is included to expose the 0.5 balanced-accuracy floor. F1’s image test score was observed before we selected the F2 model, so later reuse of this test is adaptive.

5.3 F2: one prespecified nonlinear comparison

After F1, we registered one random-forest comparison using the same 29 features and grouped split: 300 trees, `min_samples_leaf=8`, `max_features=sqrt`, `class_weight=balanced_subsample`, seed 26, and the remaining scikit-learn defaults. No new features are added. The threshold is selected using validation probabilities only. Candidate thresholds include distinct probabilities,

adjacent midpoints, and 0, 0.5, and 1. The first selection tier requires bad recall at least 0.60 and good recall at least 0.55; within an eligible tier, maximize balanced accuracy, break ties by distance to 0.5, then by higher threshold. The grouped validation set selected 0.314465497255814 in the Linux reference; the earlier macOS run selected 0.31713680975737424.

The continuation gate, written before fitting F2, required test balanced accuracy at least 0.65, bad recall at least 0.60, **and** good-image false-positive rate at most 0.45. F2 passed the first two conditions and missed the third. We stopped optimizing automatic image rejection on this repeatedly viewed test.

5.4 F3 and F4: split comparison without model search

F3 uses the same feature vector, forest configuration, and threshold rule, now with the original ZIP’s train/validation/test folders. Its validation threshold is 0.503368900036556 in the Linux reference; the earlier macOS run selected 0.4984234764442013. F4 refits both frozen configurations and scores them on the intersection of their test image IDs. F4 changes no feature, model parameter, or threshold rule. The source and grouped full-test results were already seen when F4 was chosen; F4 is a diagnostic paired comparison, not a fresh blinded validation. A second dataset would be required to test transport beyond this source.

6 Measured results

6.1 F1 and F2 on prefix-group holdout

Evaluation	Test good/bad	Balanced accuracy	Bad recall	Good images flagged bad
Always good	376/360	0.500000	0.000000	0/376
Metadata	376/360	0.523345	0.472222	160/376
logistic				
F1 image logistic	376/360	0.608836	0.802778	220/376
F2 image forest	376/360	0.657920	0.797222	181/376

Confusion matrices (true good/bad rows, predicted good/bad columns) are: always good $\begin{bmatrix} 376 & 0 \\ 360 & 0 \end{bmatrix}$; metadata $\begin{bmatrix} 216 & 160 \\ 190 & 170 \end{bmatrix}$; F1 image $\begin{bmatrix} 156 & 220 \\ 71 & 289 \end{bmatrix}$; and Linux F2 forest $\begin{bmatrix} 195 & 181 \\ 73 & 287 \end{bmatrix}$. The F2 validation set has 179 good and 73 bad images, matrix $\begin{bmatrix} 99 & 80 \\ 19 & 54 \end{bmatrix}$, and balanced accuracy 0.646399.

F1 improves balanced accuracy by 0.085491 over metadata, but falsely flags 58.51% of good test images. Linux F2 lowers that count to 181 and improves balanced accuracy by 0.049084, while its good-image false-positive rate remains $181/376 = 0.481383$. This exceeds the prespecified 0.45 ceiling, so the automatic quality-gate direction was stopped. A 2,000-resample bootstrap over its 15 test prefixes gives an approximate balanced-accuracy interval of 0.575-0.730; these prefixes are not verified chips. F2 reused a test already viewed for F1 and cannot be called a fresh blind evaluation.

6.2 Full tests under different splits

Evaluation	Test	Balanced	Good-image		
	good/bad	accuracy	Good recall	Bad recall	false positives
F3, source folders	365/291	0.799925	0.843836	0.756014	57/365
F2, prefix groups	376/360	0.657920	0.518617	0.797222	181/376

The Linux source matrix is $[[308, 57], [71, 220]]$; the grouped matrix is $[[195, 181], [73, 287]]$. Source validation has 163 good and 123 bad images, matrix $[[135, 28], [32, 91]]$, and balanced accuracy 0.784029. A 2,000-resample bootstrap over source test prefixes gives an approximate balanced-accuracy interval of 0.744-0.848.

The source-folder test has 57/57 date-like prefixes also present in source training. The grouped test has 0/15 prefixes in grouped training. The Linux full-test balanced-accuracy gap is 0.142005. However, the tests differ in composition: 505 source-test images are absent from grouped test, and 585 grouped-test images are absent from source test. The two training sets, validation sets, and selected thresholds also differ. The gap is an audit signal, not a causal estimate of the cost of overlap.

6.3 Same 151 images under both models

The test intersection has 68 good and 83 bad images from 14 date-like prefixes. It is smaller than either full test. The table below reuses those exact IDs on both sides.

Model	Balanced accuracy	Good recall	Bad recall	Good-image false positives
F3 source-split model	0.756467	0.838235	0.674699	11/68
F2 grouped-split model	0.690379	0.573529	0.807229	29/68

The source model’s shared-image matrix is $[[57, 11], [27, 56]]$; the grouped model’s is $[[39, 29], [16, 67]]$. The paired balanced-accuracy difference is $+0.066088$. Both models are correct on 92 images; only the source model is correct on 21; only the grouped model is correct on 14; both are wrong on 24. A paired 14-prefix bootstrap gives an approximate 95% difference interval of -0.042091 to $+0.122483$ (2,000/2,000 valid draws). The prespecified strong-evidence gate required a difference of at least 0.08 and an interval lower bound above zero; neither condition passed. The interval does not establish a general direction of evaluation bias.

Holding test images fixed removes test-composition differences. It does not equalize the rows used to train the forests or the rows used to choose their thresholds. It also cannot turn a date-like prefix into a physical chip ID. The result is not evidence that the same chip was used for training and test.

6.4 Cell-type boundaries

The grouped F2 full test includes A549 (151 good/81 bad), CACO (27/55), HPMEC (187/152), HSAEC (11/24), HUVEC (0/39), and NHBE (0/9). With no good images in the last two cell types, their good recall and balanced accuracy are undefined. The generated JSON retains each cell type’s denominator and confusion matrix; no single cell-type result is promoted as a separate discovery.

7 Software artifact and real workflow

The offline tool has a generic core and an adapter for this dataset. The generic command reads two JSONL record sets, validates them, and writes `audit.html` and `audit.json`. The HTML is a single local page that summarizes overlap, test denominators and errors, the shared-image comparison, and the interpretation limits. The JSON stores both input records, computed

counts, metadata, and input file hashes. The artifact does not call a model API or upload images.

For this case study, the `ooc` command rebuilds test predictions from locally extracted image features and fixed model specifications. Before export, it checks fresh F2 and F3 results against the refitted models and checks F4’s shared-image counts, confusion matrices, transitions, paired difference, and interval. On a prepared real feature cache, Linux ARM64 reproduced the independent Kaggle Linux x86-64 deterministic F2-F4 fields and exported the report with **status: verified**. This checks the reporting path; it is not a second biological dataset or a new external validation result.

A researcher with their own per-image predictions can bypass the example adapter. They supply source and grouped JSONL records, with group IDs that reflect their actual experimental provenance. The generic report then exposes group overlap and both test compositions. If the test sets have no shared IDs, it cannot compute a same-image contrast. If a test has only one class, balanced accuracy is undefined. These are intentional boundaries of the evidence.

The source-to-report route is `python run_full_audit.py`. It downloads and verifies both original Zenodo files, extracts the 3,072 image feature vectors, computes F1-F4, and writes the offline HTML, JSON, and `run_evidence.json`. On a local machine its default four-minute budget allows the same command to resume the ZIP download and feature extraction. A clean Kaggle Linux CPU run used `--time-budget 0` to complete the route in one invocation, produced the report, matched the Linux reference, and exited 0. The corresponding macOS run uses the same feature SHA-256 but different forest trees; its report marks the platform difference and the command exits 3.

8 Reliability, limitations, and application value

8.1 What is directly observed

The source folder assignments, the labels, the date-like image prefixes, and the predictions are inspectable. For those supplied identifiers, 57 of 57 source-test prefix groups also occur in source training. The different full-test scores and their denominators are observable, as are the same-image predictions on the 151-image intersection. We supply counts rather than only a single summary score so a researcher can see false alarms on good cultures.

8.2 What is not identified

A filename prefix is not independently validated as a physical chip, biological replicate, or day of acquisition. Even if the apparent date meaning were exact, images from different dates could share a chip, and images from the same date could come from different chips. The dataset does not resolve that ambiguity. The same-image result cannot isolate group overlap from a change of training data or threshold. The paired interval uses only 14 observed prefixes and crosses zero. No second non-pathogen organ-on-chip dataset with verified per-sample groups and labels has yet produced an effect estimate. Repeated attention to this source test during F1-F4 also limits how much the later numeric comparisons can be treated as confirmatory.

The quality labels assess brightfield image suitability. They do not quantify cell viability, therapeutic effect, toxicity, or clinical outcome. In particular, our false-positive counts are flags on expert-labeled good images, not measured damage to cultures. Any production triage workflow would require prospective testing, a definition of the actual acquisition unit, and user-selected costs for missed bad images versus good images sent to review.

8.3 Intended application

The audit can serve as an evaluation checklist with executable calculations. When a team has real chip IDs, it can use them as group fields and inspect whether its test chips were truly absent from training. When it has only acquisition sessions or batches, the result must carry that weaker name. The same code can also compare two test protocols on their shared images, so apparent improvements on different test sets are not casually treated as paired effects. This is a useful research workflow even when a model score does not clear a deployment gate.

9 Reproducibility and dependency disclosure

9.1 Hardware and software

The code runs on CPU with Python 3.12. `requirements.txt` pins NumPy 2.5.3, Pillow 12.3.0, SciPy 1.18.1, and scikit-learn 1.9.1. It uses no paid service, proprietary model, non-public data, or GPU. The source ZIP is roughly 6.7 GB, so obtaining all images is the largest transfer; the Python downloader saves progress and the main command defaults to a 240-second local budget. The generated feature cache and results stay in ignored local files. Runtime measurements are specific to their stated environments.

9.2 Linux reference and independent clean run

An earlier clean Kaggle Linux x86-64 CPU run started without a prepared cache, verified the 6,710,767,405-byte ZIP against source MD5 `8f7e058996203d48eb03b2d86c0a2e4d`, verified the datasheet SHA-256, and extracted features for all 3,072 image IDs. Its feature-cache SHA-256 was `65bb227546f96b7f4643ccf5f98caf50ca38b90c23cce221367dae7d5c6e5902`. That run exited 3 because the reference still contained earlier macOS measurements. Its actual Linux F2-F4 output is now the frozen Linux reference; the macOS record remains separately identified in the same file.

A second Kaggle run used the public one-command code and a new, empty `/kaggle/temp/ooc_audit_cache`. It downloaded the original files again, verified the same source and feature hashes, recomputed F1-F4, and produced `audit.html`, `audit.json`, and `run_evidence.json` with `status: verified`. The private verification job completed with exit 0 on four free Linux x86-64 CPU cores, with GPU disabled and internet access for Zenodo. The audit command took 601.00 seconds and recorded 218.98 MiB peak resident memory. The two report SHA-256 digests are recorded in `run_evidence.json` and match the downloaded artifacts. Its source test had 57/57 date-like prefixes also in training, and the shared test had 151 images, 68 good and 83 bad, from 14 prefixes.

Measure	Clean Linux reference	Original macOS measurement
Grouped F2 test balanced accuracy	0.657920	0.657801
Grouped F2 good images falsely flagged	181/376	179/376
Source F3 test balanced accuracy	0.799925	0.797882
Source F3 good images falsely flagged	57/365	61/365
Same-151-image paired balanced-accuracy difference	+0.066088	+0.070783
Paired 14-prefix bootstrap 95% interval	-0.042091 to +0.122483	-0.031747 to +0.127527

We located the first measured divergence before validation and threshold selection. On the same ordered feature and label arrays, with identical seeds and one fitting thread, the first

forest tree is identical across macOS Python 3.12 and Linux x86-64/ARM64 Python 3.12. In a 300-tree grouped fit, tree 3 first differs at node index 96: macOS selects feature 7 at threshold 0.0150570632, while Linux selects feature 9 at 0.0283526825. A 64-row probe first differs in tree 16. Linux x86-64 and ARM64 agree on every probed tree, validation probability, and threshold. The precise lower-level cause of the tree split is not established; the identical source and feature hashes rule out a different image cache as its cause.

The Linux ARM64 check reproduced 820 deterministic reference leaf fields, including 207 floating-point fields, with maximum observed difference zero. The frozen comparison allows only $1\text{e-}12$ absolute numeric difference and requires exact counts, labels, split summaries, thresholds’ nonnumeric selection fields, and confusion matrices. The feature SHA-256 must match exactly; source ZIP MD5 and datasheet SHA-256 are verified before evaluation. This tolerance is far below the measured cross-platform score and threshold differences. On macOS, the report explicitly marks matching features with platform-dependent forest scores and exits 3. A changed source or feature file likewise cannot receive `verified` status.

The grouped false-positive rate exceeds the prespecified 45% ceiling on both systems. The descriptive full-test gap exceeds the audit-direction screen of 0.10 on both systems. The paired difference is below the 0.08 stronger-evidence screen and its interval crosses zero on both systems. We do not overwrite the earlier measurements or infer a general physical-chip leakage effect from either run.

9.3 Reproduction sequence

From the public repository root, create a Python 3.12 virtual environment, install `requirements.txt`, and run `python run_full_audit.py`. The default local time budget pauses with a resumable instruction; repeat the same command until it completes. On a remote CPU without a short process limit, use `python run_full_audit.py --time-budget 0`. The script performs the spreadsheet and ZIP checks, feature extraction, F1-F4 calculations, Linux-reference comparison, and report export. The source mapper reads ZIP64 metadata, reconciles 3,072 IDs and source-folder labels to the spreadsheet, and writes the mapping locally. F4 refits both models and checks the stored F2/F3 results before the offline report is written. The JSON records input SHA-256 hashes and per-image predictions. No private database, model file, paid API, or GPU is needed.

A separate generic route accepts two user-provided record files:

```
python audit_report.py records \
```

```
--source-records SOURCE.jsonl \  
--grouped-records GROUPED.jsonl \  
--output OUTPUT_DIR
```

Each JSONL record must use a stable `id`, a provenance-backed `group`, `label` 0 or 1, `split` train/val/test, and a `prediction` on test only. The record contract lets another team run the audit without adopting our image features or random forest. Their group labels, reference labels, and predictions remain their responsibility.

10 Prior work, originality, and responsible disclosure

The image dataset and its expert quality labels predate this competition [1, 2]. The published same-data classification study also predates this work [3]. We did not create the source images, labels, or the prior paper’s model. Our competition work consists of the frozen grouped and source-split evaluations, the same-image comparison, the generic audit calculations, the offline report generator, and the reproducibility checks. All results in this report are generated by those scripts from publicly obtainable inputs. No result or demo is simulated.

The source package’s split was accepted as an object of study, not as an independent ground truth about physical chips. A finding of 57/57 prefix overlap is not a finding of chip-level data leakage. The full-test accuracy difference does not compare identical samples. The paired difference and its crossing-zero interval do not warrant a claim that source splits are systematically optimistic. We make these distinctions because the competition asks for verifiable results and because an evaluation tool loses value if its own headline overstates the evidence.

There is one individual entrant, Yan Su. Python, NumPy, Pillow, and scikit-learn are the software runtime; no foundation model or paid inference API is part of the product or the reported quantitative result. AI coding assistance may be used to write and review code and prose, but numeric results are recomputed by the published scripts and checked against per-image records. Images remain attributed to their source and are not redistributed in this repository. The code is released under MIT.

11 References

1. Movčana et al. [Organ-on-a-Chip \(OOC\) Image Dataset](#). Zenodo (2023), DOI: [10.5281/zenodo.10203721](#). Source record, downloadable data, and record-level license metadata.
2. Movčana et al. [Organ-On-A-Chip \(OOC\) Image Dataset for Machine Learning and Tissue](#)

- Model Evaluation.** *Data* 9(2), 28 (2024), DOI: [10.3390/data9020028](https://doi.org/10.3390/data9020028). Experimental context and expert image-quality labels.
3. George and Kenry. [Supervised-Learning-Driven Interrogation of Organ-on-a-Chip Quality from Microscopy Images](#). *Chemical & Biomedical Engineering* 2(12), 739-745 (2025), DOI: [10.1021/cbe.5c00087](https://doi.org/10.1021/cbe.5c00087). Its image-level split and stated need for further stability assessment motivate the distinct audit question here.
 4. [NumPy license](#). Official NumPy documentation.
 5. [Pillow license](#). Official Pillow documentation.
 6. [scikit-learn 1.9.1 release](#). Official PyPI distribution and license metadata.
 7. [CC BY 4.0 legal code](#) and [CC BY-SA 4.0 legal code](#). Creative Commons.
 8. [SciPy license](#). Official SciPy project license.

12 Appendix A: denominators and decision gates

Version	Test unit / denominator	Primary observation	Decision
F1	15 held-out date-like prefixes, 736 images	Image BA 0.608836; 220/376 good images flagged bad	One prespecified nonlinear comparison allowed
F2	Same grouped test, 736 images	Linux forest BA 0.657920; 181/376 good images flagged bad	Automatic quality gate stopped because 48.14% > 45%
F3	Original folder test, 57 overlapping prefixes, 656 images	Linux forest BA 0.799925; nonpaired gap +0.142005	Audit direction explored; paired check required
F4	Exact common test IDs, 151 images, 14 prefixes	Linux paired gap +0.066088; interval crosses zero	Strong-evidence gate failed; report as diagnostic
F7	Real indexed images	Offline HTML and JSON with per-image records	One-command report entrance accepted

The F3 candidate screen required source-test balanced accuracy at least 0.75 and full-test gap at least 0.10, both of which were observed on macOS and Linux. That screen only justified the

F4 diagnostic, not a leakage claim. The F4 strong-evidence screen required a paired gap at least 0.08 and an approximate interval lower bound above zero; neither was observed on either platform. These thresholds were used as research decisions, not post hoc significance tests.

13 Appendix B: numeric audit checks

The following equalities can be checked against the generated Linux JSON and code:

- F2 grouped full test: $195 + 181 = 376$ expert-good images; $73 + 287 = 360$ expert-bad images. Balanced accuracy is $(195/376 + 287/360)/2 = 0.6579196217$.
- F3 source full test: $308 + 57 = 365$ expert-good images; $71 + 220 = 291$ expert-bad images. Balanced accuracy is $(308/365 + 220/291)/2 = 0.7999246811$.
- Shared test: 68 expert-good plus 83 expert-bad images equals 151 IDs. The F3 matrix is $[[57, 11], [27, 56]]$; the F2 matrix is $[[39, 29], [16, 67]]$.
- Shared-image F3 balanced accuracy is $(57/68 + 56/83)/2 = 0.7564670446$; F2 is $(39/68 + 67/83)/2 = 0.6903791637$; their difference is 0.0660878809.
- Correctness transitions sum to the common denominator: 92 both correct + 21 source-only correct + 14 grouped-only correct + 24 both wrong = 151.
- Test-composition accounting: 656 source test - 151 common = 505 source-only; 736 grouped test - 151 common = 585 grouped-only.

The audit program computes these numbers from records. The appendix shows how to check them by hand; it is not a separate source of values. Full precision, source hashes, and per-image assignments are held in the generated local `audit.json`. The HTML rounds for reading, and the JSON preserves the computed values.